

Comparaison Structure Base de Données - JobbingTrack

[← Retour au README principal](#)

Ce document compare la **spécification fonctionnelle** fournie avec la **structure actuelle** des schémas Prisma dans le projet JobbingTrack.

Synthèse Générale

✓ Points de Convergence

- **Modèle User** : Structure cohérente avec tous les champs requis
- **Modèle Candidature** : Correspondance quasi-parfaite avec les relations attendues
- **Modèles Entreprise, Contact, Relance, Entretien** : Structure de base alignée
- **Relations many-to-many** : Tables de jonction correctement implémentées

⚠ Principales Divergences

- **Architecture microservices** : Schémas dupliqués et inconsistants entre services
 - **Modèles manquants** : HistoriqueEtatCandidature, SyncQueue partiellement implémentés
 - **Relations polymorphes** : Événements correctement liés mais structure complexe
 - **Modèles de sécurité** : Duplication entre schéma principal et security-service
-

Architecture des Schémas

Schéma Principal (`backend/prisma/schema.prisma`)

Statut :  **COMPLÈTE** - Référence unique et exhaustive

Contenu : 25+ modèles avec toutes les relations, tables de jonction, et modèles de sécurité

- Relations many-to-many complètes
- Modèles de sécurité intégrés
- Historique et synchronisation
- Gestion d'archivage

Services Spécialisés - État Inconsistant

✓ Auth Service (`backend/auth-service/prisma/schema.prisma`)

Statut :  **SIMPLIFIÉ** - Version réduite fonctionnelle

- **Avantages** : Schéma allégé, rapide à charger
- **Limites** : Relations many-to-many absentes, modèles de sécurité manquants

✓ Application Service (backend/application-service/prisma/schema.prisma)

Statut : ⚠ **FOCALISÉ** - Modèles métier uniquement

- **Avantages** : Optimisé pour les candidatures
- **Limites** : Schéma très restrictif, manque les relations

⚠ Services avec Schémas Minimalistes

Liste : call-service, event-service, interview-service, followup-service, workflow-service

- **Contenu** : HealthCheck uniquement
- **Problème** : Services non fonctionnels pour la persistance

✓ Security Service (backend/security-service/prisma/schema.prisma)

Statut : ⚠ **SPÉCIALISÉ** - Modèles de sécurité uniquement

- **Avantages** : Focus sur la sécurité
- **Problème** : Duplication avec schéma principal



Analyse Détaillée par Modèle

1. User - ✓ COHÉRENT

Spécification vs Réalité :

```
+ id, email, password, firstName, lastName, phone ✓
+ profilePicture, role, roles[] ✓
+ resetToken, resetTokenExpiry ✓
+ isActive, timestamps ✓
+ relations: candidatures, contacts, relances ✓
- nom, prenom (utilise firstName, lastName)
```

2. Candidature - ✓ ALIGNÉ

Spécification vs Réalité :

```
+ Tous les champs métier présents ✓
+ Relations: user, entreprise, plateforme ✓
+ Relations: relances, appels, entretiens ✓
+ Relations: evenements, documents ✓
```

- + Relations: historique etats ✓
- + Relations: contacts (many-to-many) ✓

3. Entreprise - ✓ CORRECT

Spécification vs Réalité :

- + nomEntreprise → name ✓
- + secteurActivite → industry ✓
- + tailleEntreprise → size ✓
- + Relations avec candidatures ✓
- + Relations many-to-many avec contacts ✓

4. Contact - ✓ CONFORME

Spécification vs Réalité :

- + Structure complète ✓
- + Relations many-to-many avec entreprises ✓
- + Relations many-to-many avec candidatures ✓
- + Relations avec appels, entretiens ✓

5. Relance - ✓ RESPECTÉ

Spécification vs Réalité :

- + Relations avec candidatures et entreprises ✓
- + Relations many-to-many avec contacts ✓
- + Relations avec appels et événements ✓

6. Appel - ✓ NOUVEAU - BIEN IMPLÉMENTÉ

Modèle ajouté par rapport à la spécification :

- + Modèle complet avec tous les statuts
- + Relations avec contacts, candidatures
- + Gestion des appels entrants/sortants
- + Intégration avec événements

7. Entretien - ✓ COMPLET

Spécification vs Réalité :

- + Relations many-to-many avec contacts ✓
- + Gestion des types et statuts ✓
- + Intégration calendrier ✓

8. Evenement - ✓ POLYMORPHE - BIEN FAIT

Spécification vs Réalité :

- + Relations polymorphes correctement implémentées ✓
- + Contraintes d'unicité respectées ✓
- + Un seul lien actif par événement ✓

9. HistoriqueEtatCandidature - ⚠ PARTIEL

Spécification : HistoriqueEtatCandidature

Réalité : ApplicationStatusHistory

- + Structure cohérente ✓
- + Relations correctes ✓
- Nom différent mais fonction équivalente

10. Notification - ✓ ÉTENDU

Spécification vs Réalité :

- + Structure de base respectée ✓
- + Champs additionnels (contactId, eventId) ✓
- + Support multicanal ✓

11. Document - ✓ AMÉLIORÉ

Spécification vs Réalité :

- + Structure de base ✓
- + Gestion des versions (ApplicationDocument) ✓
- + Support multi-applications ✓

12. SyncQueue - ✓ PRÉSENT

Spécification vs Réalité :

- + Modèle de synchronisation ✓
- + Gestion des tentatives ✓
- + Support offline ✓

Relations Many-to-Many - ✓ COMPLÈTES

Tables de Jonction Implémentées :

1. **ContactEntreprise** (ContactCompany) ✓
2. **ContactCandidature** (ContactApplication) ✓
3. **RelanceContact** (FollowUpContact) ✓
4. **EntretienContact** (InterviewContact) ✓
5. **ContactEvenement** (ContactEvent) ✓

Bonus : Table `ApplicationDocument` pour gestion des versions

Sécurité - ⚠️ DUPLICATION

Schéma Principal :

- SecurityLog, Vulnerability, IntrusionAttempt
- DDoSAttack, SecurityAlert, SecurityMetric

Security Service :

- Modèles similaires mais structure différente
- **Risque** : Inconsistance des données

Recommandation :

- Centraliser la sécurité dans le schéma principal
- Supprimer la duplication

Synchronisation - ✓ BIEN PENSÉ

Fonctionnalités :

- **SyncQueue** : File d'attente de synchronisation

- **EntityHash** : Détection des modifications
 - **Archivage soft** : Conservation des données
 - **Multi-device** : Support offline
-

Évaluation Globale

Points Forts

1. **Modèle fonctionnel complet** dans le schéma principal
2. **Relations complexes** correctement implémentées
3. **Gestion d'archivage** sophistiquée
4. **Support multilingue** (enumérations)
5. **Synchronisation offline** bien pensée

Points d'Attention

1. **Architecture microservices incomplète** - Services non fonctionnels
2. **Duplication des schémas** entre services
3. **Inconsistance** entre services spécialisés
4. **Maintenance** complexifiée par la duplication

Recommandations

Priorité Haute :

1. **Choisir une architecture** : Soit monolithique (schéma principal), soit microservices complets
2. **Nettoyer les schémas** des services non utilisés
3. **Uniformiser** les modèles entre services actifs

Priorité Moyenne :

1. **Documenter** clairement l'architecture choisie
2. **Migrer** les données des services vers l'architecture principale
3. **Supprimer** la duplication sécurité

Priorité Basse :

1. **Optimiser** les requêtes avec les index appropriés
 2. **Documenter** les conventions de nommage
-

Plan de Migration Suggéré

Phase 1 : Stabilisation (1-2 semaines)

- Décider de l'architecture : **Recommandation : Schéma principal**
- Nettoyer les schémas de services
- Migrer les données des services vers schéma principal

Phase 2 : Optimisation (1 semaine)

- Ajouter les index manquants
- Optimiser les requêtes complexes
- Documenter les performances

Phase 3 : Maintenance (Continue)

- Établir des conventions claires
- Automatiser les migrations
- Surveiller les performances

Conclusion : Le modèle de données est **fonctionnellement complet** et **bien conçu**. Le principal défi est **architectural** : consolider vers une approche cohérente plutôt que de maintenir plusieurs schémas partiels.